ARE 212 Section 2:
`knitr` and Matrix Operations

Fiona Burlig

January 28, 2016

This week, we'll take a hiatus from working with real data to learn about matrix operations in `R`. Since you're prohibited from using canned routines in this class, these will be your essential building blocks. Before we take the red pill, though, we'll go through a gentle introduction to `knitr`.

`knitr` intro

`knitr` is a form of dynamic document in `R`, making it possible to embed `R` code and output into LaTeX documents. Change a dataset at the beginning of your code? Re-compile your `knitr` document and the resulting output will update automatically. `knitr` also lets you elegantly combine code and text, which will be especially useful for problem sets and exams - it's Max and my preferred format for work in this class.

As I mentioned at the end of last week, you'll need to have a LaTeXdistribution and a PDF reader installed to use `knitr`. If you don't have a LaTeXdistribution installed yet, you'll be able to follow along, but probably not use `knitr` today - downloading LaTeXtakes a while.[1]

Once we've got everything installed, we need to do two things in `Rstudio` to enable `knitr`. First, install the `knitr` package, just like we did with `haven` and `dplyr` last week:

```
install.packages("knitr")
```

Next, go to [Mac:] `Rstudio > Preferences` [PC:] `Tools > Global Options`. Click on the `Sweave` tab. Make sure the **Weave Rnw files using:** field is set to `knitr`, and that the **Typeset LaTeXfiles into pdf using** field is set to `pdfLatex`. Great. Now we're good to go! Open a new .Rnw file in `Rstudio` (go to `File > New File > R Sweave`). Copy and paste the following preamble into your new file:[2]

```
\documentclass[11pt]{article}
\usepackage[margin = 0.5in]{geometry}
\setlength{\parindent}{0in}
\usepackage{verbatim}
\usepackage{bm}
\usepackage{hyperref}
```

[1] Mac users don't need to get a new PDF reader - the built in one is fine. If you're a Windows user without a PDF reader, go ahead and install `Sumatra` from `http://www.sumatrapdfreader.org/free-pdf-reader.html`.

[2] `knitr` can use any LaTeXpreamble you like. After this section, go ahead and use your existing preamble to set up your document just the way you like it.

```
\author{YOUR NAME}
\title{\texttt{knitr} and me:\\ The beginning of a beautiful friendship}
\begin{document}
\maketitle
\end{document}
```

To set up your `knitr` file so that things look nice, go ahead and include this in the preamble (I like to put it before the call):

```
<<"knitrSetup", eval=TRUE, echo=FALSE>>=
# load the knitr library
library(knitr)
#sets width for knitr so your text doesn't run off the page
options(width=65)
@
```

This document should now be able to compile (click on the `Compile PDF` button, or hit [Mac:] `Command + Shift + K` [Windows:] `Ctrl + Shift + K`). One important note before we get started: `knitr` strongly discourages setting a working directory - it uses the directory in which the .Rnw file is placed as its working directory. You can still use relative paths from there, but going up and out is hard. This might seem annoying, but actually encourages good file management practices. All you need to know is: keep files you're going to use with a `knitr` document either in the same folder as your .Rnw file or in a subfolder thereof.[3]

1 Matrix 101: (1999)

`R` stores data in a bunch of different ways. We'll mainly be using three types of objects in this course: vectors, matrices, and dataframes (or their better selves which we saw last week, data frame tbls). Let's kill two birds with one stone and explore `knitr` and matrices at the same time. First, add a `knitr` "chunk" to your document with [Mac:] `Command + Option + I` [Windows:] `Ctrl + Alt + I`. This will add the following snippet of code to your document:

```
<<eval = TRUE, echo = TRUE>>=


@
```

This is a `knitr` chunk. You can name each chunk, and also set values for two key options: `echo` and `eval`.[4] For the purposes of this section, let's leave both `echo` and `eval` as `TRUE`. Note that `knitr` runs in its own instance of `R` - when you compile a `knitr` document, your code will run and output to PDF, but it won't affect your workspace/console. But you can run your `knitr` chunks in the console by clicking in a chunk and hitting [Mac:] `Ctrl + Opt + C` [PC:] `Ctrl + Alt + C`.
Now that we've got the basics down, let's do some matrix work in `knitr`. We'll start with vectors. Making a vector in `R` is super easy. Name your chunk "vectors" and set `echo` and `eval` to `TRUE` like this:

[3] ARE 212 section doesn't, on principle, end sentences with prepositions. Boom. (But don't be a snob about this: you don't want to be mistaken for a Harvard student.)

[4] There are a couple more, but not so useful, so we'll leave that for your own exploration. Both are set `=TRUE` by default. Setting `echo = FALSE` will prevent your chunk from being displayed in your PDF; setting `eval = FALSE` will prevent your code from actually running.

```
<<"vectors", eval = TRUE, echo = TRUE>>=


@
```

Ok, good, but that's kind of boring. Let's do something with it. Insert the following code between the `>>=` and the `@`:

```
vec <- 1:5

print(vec)
```

Run the chunk, and you should see:

```
vec <- 1:5

print(vec)

## [1] 1 2 3 4 5
```

Awesome! We've made our first vector. Let's add a second vector, this time of strings, to our code snippet.

```
# the 1:5 command tells R to make a vector with all the numbers from 1 to 5, in order
vec <- 1:5

print(vec)

## [1] 1 2 3 4 5

vecStr <- c("My", "GSI", "is", "awesome;", "8", "AM", "section", "isn't.")

vecStr

## [1] "My"       "GSI"      "is"       "awesome;" "8"
## [6] "AM"       "section"  "isn't."
```

See the `c()` command? It joins multiple objects together into a vector. We can also call specific elements of a vector if we want to (note that `R` vector indices start at 1, not 0):

```
myWord <- vecStr[4]
myWord

## [1] "awesome;"
```

Time to move on to the vector's two-dimensional cousin, the matrix. Start a new code snippet and call it `"matrix"`.[5] Let's build some matrices. We'll start with a matrix **A**.

[5] So. Freaking. Original.

```
(A <- matrix((data = 1:6), ncol = 2))
```

```
##      [,1] [,2]
## [1,]    1    4
## [2,]    2    5
## [3,]    3    6
```

```
#Notice that by putting parentheses around the command
#  we get R to print the output without having to ask. Nice.
```

Now I want to (manually) build matrix $\mathbf{B}$, which will be $\mathbf{A}'$. Let's try:

```
(B <- matrix((data = 1:6), ncol = 3))
```

```
##      [,1] [,2] [,3]
## [1,]    1    3    5
## [2,]    2    4    6
```

Oops. That's not quite right. Let's try again:

```
(B <- matrix((data = 1:6), ncol = 3, byrow = TRUE))
```

```
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6
```

Much better. The `byrow = TRUE` tells `R` to cycle through rows rather than columns while placing data. Be aware of how you set up your matrices in the future! For a 2-by-3 matrix, it's easy to visually make sure that `B` is in fact the transpose of $\mathbf{A}$. But it would be nice to get `R` to tell us, rather than making us use our own brainpower.

```
#note both the logical and the "transpose" function t()
B == t(A)
```

```
##      [,1] [,2] [,3]
## [1,] TRUE TRUE TRUE
## [2,] TRUE TRUE TRUE
```

`R` will tell us, element by element, whether $\mathbf{B}$ is indeed equal to $\mathbf{A}'$. But that's also kind of annoying. I want one answer:

```
all(B == t(A))
```

```
## [1] TRUE
```

Et voila. It's also convenient to be able to easily ask `R` for the dimensions of any matrix:

```
dim(A)
```

```
## [1] 3 2
```

```
dim(B)

## [1] 2 3
```

2 Matrix 201: Reloaded (2003)

Okay, that was a less-than-amazing movie. But matrix operations in R are actually great.[6]

Matrix multiplication in R uses the command %*%, whereas scalar multiplication uses *. Since **A** is a 3-by-2 matrix and **B** is a 2-by-3 matrix, we should be able to premultiply **A** by **B**. Let's try:

```
B %*% A

##      [,1] [,2]
## [1,]   14   32
## [2,]   32   77
```

We shouldn't be able to premuliply $\mathbf{A}'$ by **B**, though. Verify:

```
B %*% t(A)

## Error in B %*% t(A): non-conformable arguments
```

Good, R returned an error. That's reassuring. We can, however, use scalar multiplication on two matrices of the same dimension - this will multiply each element in **B** with the element in the same position from $\mathbf{A}'$ - this is called taking the Hadamard product, and is denoted $\mathbf{B} \circ \mathbf{A}'$. In R:

```
B * t(A)

##      [,1] [,2] [,3]
## [1,]    1    4    9
## [2,]   16   25   36
```

We can also do more innocuous scalar multiplication. Let's multiply every element in **A** by 2:

```
A * 2

##      [,1] [,2]
## [1,]    2    8
## [2,]    4   10
## [3,]    6   12
```

[6] Here's the part of the class where you start being *really* glad that we're not doing this in Stata. Don't get me wrong - I like Stata a lot. But doing stuff like this in Stata is a Huge. Pain.

3 Matrix 301: Revolutions (2003)

This was the *really* bad one. We're almost out of the woods, but before we're done, I want to show you a couple additional matrix tools in `R` that might be helpful.

We'll use the identity matrix - an n by n matrix whose diagonal elements are all equal to one and off-diagonal elements are all equal to zero - quite a bit in this course. Making one is easy in `R`. Let's make a 5-by-5 identity matrix, $\mathbf{I}$:

```
(I <- diag(5))
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    0    0    0    0
## [2,]    0    1    0    0    0
## [3,]    0    0    1    0    0
## [4,]    0    0    0    1    0
## [5,]    0    0    0    0    1
```

It'd better be the case that $\mathbf{I} = \mathbf{I}^{-1}$. Let's check:

```
#solve() inverts matrices. Take that, college linear algebra class.
all(I == solve(I))
```

```
## [1] TRUE
```

It can also be useful to find the trace (sum of the diagonal elements) of a matrix. This time, `R` doesn't actually have a built-in command - but thankfully, somebody has written a package with a trace function in it. Let's grab and load that package now:

```
install.packages('psych')
```

```
library(psych)
```

We can finally calculate the trace of $\mathbf{I}$:

```
tr(I)
```

We can also take the determinant of matrices using `R` - but remember that you can only take the determinant of a square matrix:

```
C <- matrix(1:9, ncol = 3)

det(C)
```

```
## [1] 0
```

We can also grab the eigenvalues and eigenvectors of a square matrix:

```
eigenvec <- eigen(C)$vectors
eigenvec
```

```
##            [,1]       [,2]       [,3]
## [1,] -0.4645473 -0.8829060  0.4082483
## [2,] -0.5707955 -0.2395204 -0.8164966
## [3,] -0.6770438  0.4038651  0.4082483
```

```
eigenval <- eigen(C)$values
eigenval
```

```
## [1]  1.611684e+01 -1.116844e+00 -5.700691e-16
```

Finally, there are a few situations in which you might want to add an extra column to a matrix. The easiest way to do this is with the `rbind()` and `cbind()` commands.

```
d <- 2:4
e <- 2:5
column_bound <- cbind(C, d)
column_bound
```

```
##            d
## [1,] 1 4 7 2
## [2,] 2 5 8 3
## [3,] 3 6 9 4
```

```
row_bound <- rbind(column_bound, e)
row_bound
```

```
##         d
##   1 4 7 2
##   2 5 8 3
##   3 6 9 4
## e 2 3 4 5
```

We've now added both a column and a row to a matrix.

Pro tip: For the most part, you should be keeping data in `tbl_df` format. We can also use commands very similar to `rbind()` and `cbind()` with `dplyr`. Let's give this a try.

Step 1: Load `dplyr` and create a `tbl_df` object, as well as a second object to add on.

```
library(dplyr)
```

```
##
## Attaching package:  'dplyr'
## The following objects are masked from 'package:stats':
##
##    filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union

# turn matrix C into a dataframe (and then tbl_df)
dataObject <- as.data.frame(C)
dataObject <- tbl_df(dataObject)
forCombine <- 1:3
```

Now we can combine these two objects using the `mutate` command from `dplyr`:

```
combined_data <- mutate(dataObject, newcol = forCombine)
#visualize
combined_data

## Source: local data frame [3 x 4]
##
##      V1    V2    V3 newcol
##   (int) (int) (int)  (int)
## 1     1     4     7      1
## 2     2     5     8      2
## 3     3     6     9      3

# Alternatively
combine_df <- as.data.frame(forCombine)
# We have to turn forCombine into a dataframe before binding
combined_data2 <- bind_cols(dataObject, combine_df)
combined_data2

## Source: local data frame [3 x 4]
##
##      V1    V2    V3 forCombine
##   (int) (int) (int)      (int)
## 1     1     4     7          1
## 2     2     5     8          2
## 3     3     6     9          3
```

We can also add rows to a dataframe using `dplyr`:

```
stacked_data <- bind_rows(combined_data, combined_data2)
stacked_data

## Source: local data frame [6 x 5]
##
##      V1    V2    V3 newcol forCombine
##   (int) (int) (int)  (int)      (int)
## 1     1     4     7      1         NA
## 2     2     5     8      2         NA
## 3     3     6     9      3         NA
```

```
## 4     1     4     7     NA          1
## 5     2     5     8     NA          2
## 6     3     6     9     NA          3
```

Excellent.[7]

4 A random walk

Before we leave off, I want to quickly introduce you to R's tools for simulating random data. This will be useful for problem sets, exams, and research.

Pro tip: Before doing anything involving randomness, it's a good idea to set a seed. This will make your code reproducible.

```
set.seed(12345)
```

First things first: let's take a random sample of a vector.[8]

```
#create a new vector containing the numbers 1:20
vec <- 1:20

#grab 10 random numbers from vec
vec_sampled <- sample(vec, size = 10)
vec_sampled

##  [1] 15 17  4 13  9 11  5 10 20  3

#grab 10 random numbers from vec (with replacement)
vec_sampled_rep <- sample(vec, size = 10, replace = TRUE)
vec_sampled_rep

##  [1]  4 20 13 12  8  6 17  3 20  7
```

What if we want something a little more complicated? We can simulate a bunch of coinflips using the following code:[9]

```
coinflips <- sample(c(0,1), size = 1000, replace = TRUE, prob = c(0.5, 0.5))
# How many came up heads?
sum(coinflips)

## [1] 481
```

We can also do random draws from a variety of different distributions. The syntax is similar for each, so I'll show you the Normal distribution in section and let you explore the others on their own. Let's create a vector with 100 draws from a Normal distribution.

[7] These data are *not* tidy. Check out `tidyr` to fix this with the `unite()` function. We'll get there - but not for a little while.

[8] We can also randomly sample from character vectors. Cool party trick, but less useful for this class. Please do not ever do this at a party.

[9] If you want to get some money out of your friends at the aforementioned party, run the following code with `prob` set away from 0.5, and lure them into a bet. ARE 212 does not condone betting. But if you do pull this off, want to share some of the profits?

```r
(normalsample <- rnorm(100))
```

```
##   [1]  1.07484417  1.02834779  1.39766674 -1.77545151  0.42978328
##   [6]  1.30026792 -0.06335585  0.15049032 -0.51463097  2.09940405
##  [11]  1.73618896 -0.90144677  0.29959051 -1.37031932 -1.23107895
##  [16]  0.04551297 -1.25481918 -1.60976750 -0.86619921 -1.25541605
##  [21] -1.38892013  0.36248171 -0.05746232  0.95115349  1.42720914
##  [26]  0.37839604  0.32431357  1.17831955 -1.27595786 -1.83289670
##  [31] -0.72820772  1.75747288 -0.20902481  0.35737277  0.10054034
##  [36]  1.48018852  0.44486323  0.85688338 -0.43158048 -1.21284534
##  [41]  1.69500798  1.27506986  0.89748386  0.10937531  0.24065707
##  [46]  0.97044658 -0.86804180 -1.24742254 -1.21570235 -0.84822093
##  [51] -1.02785754 -1.05627877  1.22003319  0.13280397  0.53526162
##  [56]  0.51311832  1.43676536 -1.18918465  2.40787349 -0.07200104
##  [61]  0.48476123 -1.60413993  0.56174579  0.20343980  1.31875447
##  [66] -0.77571492 -0.23023667  0.34000628  1.19521782 -0.09432936
##  [71]  0.85378954 -0.51348578 -1.43629492  1.28156303 -0.37579726
##  [76] -0.50739661  0.14959775 -1.75435447 -0.76005259 -1.20997354
##  [81]  1.33574863  0.02595279 -1.41388104 -2.13306216 -1.23769878
##  [86]  0.36861502 -0.38516190 -1.19942375  0.18363657  1.12904083
##  [91]  0.29739720  0.19856461 -0.05865534  1.04331254  1.78092678
##  [96] -1.33491673  0.53580253  0.74164060 -0.36588702  1.48275879
```

```r
# The base command has mean = 0, SD = 1, but we can do more:
(normalsample_params <- rnorm(100, mean = 100, sd = 25))
```

```
##   [1]  91.25502 102.17210  98.30384  91.80063  92.94755  85.92868
##   [7]  43.26564 112.13896 152.98793  61.32088  88.59128 129.90611
##  [13] 131.64494 114.31492 110.38257  90.18550 104.54013 115.02346
##  [19]  94.53181  89.90265 125.73145 120.88797 147.87214  95.15515
##  [25] 152.04938 131.92136 142.22647  68.15232 115.38889  87.55879
##  [31]  93.65834  96.83069 130.31919  95.64209  98.07185 106.53653
##  [37] 113.14295  74.01964 114.89562 118.26919  80.20450 106.81788
##  [43]  84.62367 123.87402 111.12508  97.57686 113.91912 108.78636
##  [49]  82.96782 146.13773 133.34353 124.88514  47.68705 141.31355
##  [55] 120.25175  84.39703  81.87091 107.77255  28.43264 140.42692
##  [61]  71.19412  84.10288  96.42563 111.20428 114.04374  93.51425
##  [67]  48.35723 106.16505 107.58377  63.21439 104.15662 107.47305
##  [73] 131.48939  98.17990  64.63414  86.21644  65.25115 102.56201
##  [79] 122.41508  70.99289 103.97600 115.21079  94.61317 178.83775
##  [85]  64.90120  43.68920 134.80024 133.31452  86.15124 107.30013
##  [91] 104.01447  85.26369  88.67571  89.84174 149.15530 122.90110
##  [97] 131.17345  86.39647  83.71264 107.06344
```

R's simulation functionality is extremely powerful - I recommend that you spend some time playing around with it.

5 Linear algebra puzzles

1. Define vectors $\mathbf{x} = [12\ \ 3]'$, $\mathbf{y} = [2\ \ 3\ \ 4]'$, and $\mathbf{z} = [3\ \ 5\ \ 7]$.Define $\mathbf{W} = [\mathbf{x}\ \ \mathbf{y}\ \ \mathbf{z}]$. Calculate $\mathbf{W}^{-1}$. If you cannot take the inverse, explain why not and adjust $\mathbf{W}$ so that you *can* take the inverse. *Hint*: the `solve()` function will return the inverse of the supplied matrices.

2. Show, somehow, that $(\mathbf{X}')^{-1} = (\mathbf{X}^{-1})'$.

3. Generate a 3×3 matrix $\mathbf{X}$, where each element is drawn from a standard Normal distribution. Let $\mathbf{A} = \mathbf{I}_3 - \frac{1}{3}\mathbf{B}$ be a demeaning matrix, with $\mathbf{i}$ a 3×3 matrix of ones. First show that $\mathbf{A}$ is idempotent and symmetric. Next show that each row of the matrix $\mathbf{XA}$ is the deviation of each row in $\mathbf{X}$ from its mean. Finally, show that $(\mathbf{XA})(\mathbf{XA})' = \mathbf{XAX}'$, first through algebra and then `R` code.

4. Demonstrate from random matrices that $(\mathbf{XYZ})^{-1} = \mathbf{Z}^{-1}\mathbf{Y}^{-1}\mathbf{X}^{-1}$.

5. Let $\mathbf{X}$ and $\mathbf{Y}$ be square 20×20 matrices. Show that $tr(\mathbf{X} + \mathbf{Y}) = tr(\mathbf{X}) + tr(\mathbf{Y})$.

6. Generate a diagonal matrix $\mathbf{X}$, where each element on the diagnonal is drawn from $U[10, 20]$. Now generate a matrix $\mathbf{B}$ s.t. $\mathbf{X} = \mathbf{BB}'$. *Hint*: There is a method in `R` that makes this easy. Does the fact that you can generate $\mathbf{B}$ tell you anything about $\mathbf{X}$?

7. Demonstrate that for any scalar c and any square matrix $\mathbf{X}$ of dimension n that $\det(c\mathbf{X}) = c^n \det(\mathbf{X})$.

8. Demonstrate that for an $m \times m$ matrix $\mathbf{A}$ and a $p \times p$ matrix $\mathbf{B}$ that $\det(\mathbf{A} \otimes \mathbf{B}) = \det(\mathbf{A})^p \det(\mathbf{B})^m$. *Hint*: Note that $\otimes$ indicates the Kronecker product[10]. Google the appropriate `R` function.

[10]The Kronecker product is a useful mathemagical tool for econometricians, allowing us to more easily describe block-diagonal matrices for use in panel data settings. I wouldn't lose sleep over it, though.